GEMPRO Notebook

GEMPRO

Contest

template.cpp

Template base para abrir um problema e começar a implementar.

```
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = (a); i < (b); ++i)

mt19937 rng(random_device{}());

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.bashrc

Atalhos de shell para compilar, rodar e comparar saídas rápido.

```
c() { g++ $1.cpp -o $1 -std=c++20; }
cs() {
    g++ $1.cpp -o $1 -std=c++20 \
        -fsanitize=address,undefined \
        -fsanitize=signed-integer-overflow -ggdb
}
gen() {
    local N=${1:-10}
    for i in $(seq 1 $N); do
        python3 gen.py > $i.in
        if [[ -f brute ]]; then
            ./brute < $i.in > $i.ans
        fi
    done
}
rr() {
    for i in $(ls *.in); do
        local f=$(basename $i .in)
        ./$1 < $f.in > $f.out
        if [[ -f $f.ans ]]; then
            if ! cmp -s $f.out $f.ans; then
                echo WA
                return 1
            fi
        fi
    done
    echo AC
}
```

Template de stress testing

Esqueleto para gerar testes aleatórios em stress testing.

```
from random import randrange as rr, randint as ri, shuffle as
↪  sh, sample as sm
from string import ascii_lowercase as alpha
from itertools import combinations as combs

def out(*a): print(" ".join(map(str, a)))

def outa(a): print(" ".join(map(str, a)))

def tree(n): return [[rr(1, i), i] for i in range(2, n + 1)]

def graph(n, m): return sm(list(combs(range(1, n + 1), 2)), m)

def arr(n, l = 0, r = 10 ** 9): return [ri(l, r) for i in
↪  range(n)]

def darr(n, l = 0, r = 10 ** 9): return sm(range(l, r), n)

def perm(n): p = list(range(1, n + 1)); sh(p); return p
```

.vimrc

Configuração enxuta do Vim para contest.

```
syntax on
set cin ts=4 sw=4 udf cul nu is

ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
↪  md5sum \| cut -c -6
```

Matemática

Algoritmo Euclideano

`egcd(a, b, x, y)` retorna o gcd e preenche Bézout em $O(\log(\min(a, b)))$.

b4f64d - math/egcd.cpp - 9 lines

```
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = egcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
} // b4f64d
```

Raiz quadrada módulo `mod`

`sqrtMod(a)` retorna a menor raiz de `x²` = `a`, ou `-1`; $O(\log^2 mod)$.

b04743 - math/mod-sqrt.cpp - 19 lines

```
int sqrtMod(int a) { // mod prime
    if (a == 0 || mod == 2) return a;
    if (powm(a, (mod - 1) / 2) != 1) return -1;
    if (mod % 4 == 3) return powm(a, (mod + 1) / 4);
```

```
    int s = __builtin_ctz(mod - 1), q = (mod - 1) >> s, z = 2;
    while (powm(z, (mod - 1) / 2) != mod - 1) z++;
    ll c = powm(z, q), x = powm(a, (q + 1) / 2), t = powm(a,
    ↪  q);
    for (int m = s; t != 1; ) {
        ll tt = t;
        int i = 0;
        while (tt != 1) tt = tt * tt % mod, i++;
        ll b = powm(c, 1 << (m - i - 1));
        x = x * b % mod;
        c = b * b % mod;
        t = t * c % mod;
        m = i;
    }
    return min<ll>(x, mod - x);
} // b04743
```

Soma de Floor

Somas com piso: `floorSum`, `floorSumI` e `floorSumSq`; custo logarítmico.

84ad33 - math/floor-sum.cpp - 45 lines

```
using i128 = __int128_t;

i128 floorSumSq(ll n, ll m, ll a, ll b);

i128 floorSum(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * n * (n - 1) / 2 + (i128)qb * n
            + floorSum(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * (n - 1) - floorSum(k, a, m, m - b - 1);
} // 440661

i128 floorSumI(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    i128 s1 = (i128)n * (n - 1) / 2, s2 = (i128)n * (n - 1) *
    ↪  (2 * n - 1) / 6;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * s2 + (i128)qb * s1
            + floorSumI(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * s1
        - (floorSumSq(k, a, m, m - b - 1) + floorSum(k, a, m,
        ↪  m - b - 1)) / 2;
} // dd026c
i128 floorSumSq(ll n, ll m, ll a, ll b) {
    if (!n) return 0;
    i128 s1 = (i128)n * (n - 1) / 2, s2 = (i128)n * (n - 1) *
    ↪  (2 * n - 1) / 6;
    if (a >= m || b >= m) {
        ll qa = a / m, qb = b / m;
        return (i128)qa * qa * s2 + 2 * (i128)qa * qb * s1 +
        ↪  (i128)qb * qb * n
            + 2 * (i128)qa * floorSumI(n, m, a % m, b % m)
            + 2 * (i128)qb * floorSum(n, m, a % m, b % m)
            + floorSumSq(n, m, a % m, b % m);
    }
    ll k = ((i128)a * (n - 1) + b) / m;
    if (!k) return 0;
    return (i128)k * k * (n - 1)
        - 2 * floorSumI(k, a, m, m - b - 1)
        - floorSum(k, a, m, m - b - 1);
} // eb1940
```

CRT

`crt(r1, m1, r2, m2)` combina duas congruências em tempo logarítmico.

6e0e1c - math/crt.cpp - 12 lines

```
// x = r1 (mod m1), x = r2 (mod m2) -> x = r (mod m)
// returns {r, m} or {0, -1} if incompatible
// chain calls to combine more congruences
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {0, -1};
    ll k = m2 / g, m = m1 / g * m2;
    ll q = ((r2 - r1) / g % k + k) % k;
    x = ((x % k) + k) % k;
    ll r = r1 + (__int128)m1 * (q * x % k);
    return {(r % m + m) % m, m};
} // 6e0e1c
```

Base de XOR

`add`, `minXor` e `maxXor`; $O(B)$ por operação.

0e002d - math/xor-basis.cpp - 20 lines

```
struct XorBasis {
    static const int B = 63; // works for nonnegative ll
    ↪  values < 2^63
    ll b[B] = {};
    bool add(ll x) {
        for (int i = B - 1; i >= 0; i--) if (x >> i & 1) {
            if (!b[i]) return b[i] = x, 1;
            x ^= b[i];
        }
        return 0;
    } // 9dfd7d

    ll minXor(ll x) {
        for (int i = B - 1; i >= 0; i--) x = min(x, x ^ b[i]);
        return x;
    } // f6abc2
    ll maxXor(ll x = 0) {
        for (int i = B - 1; i >= 0; i--) x = max(x, x ^ b[i]);
        return x;
    } // e00cc1
};
```

Teste de primalidade

`isPrime(n)` para inteiros de 64 bits em $O(\log n)$.

b5a468 - math/primality.cpp - 25 lines

```
using i128 = __int128;

bool checkPrime(ll p, ll a) {
    ll k = p - 1, d = 1;
    while (~k & 1) k >>= 1;
    for (ll e = k; e; e >>= 1) {
        if (e & 1) d = i128(d) * a % p;
        a = i128(a) * a % p;
    }
    if (d == 1 || d == p - 1) return true;
    while ((k <<= 1) < p - 1) {
        d = i128(d) * d % p;
        if (d == p - 1) return true;
    }
    return false;
} // d5cf5e

bool isPrime(ll p) {
    if (p == 1) return false;
    for (int i: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    ↪  {
```

```
        if (p == i) return true;
        if (!checkPrime(p, i)) return false;
    }
    return true;
} // a137c8
```

Fatoração de inteiros

`factor(n, fs)` adiciona em `fs` os fatores primos de `n`; usa `isPrime`.

cff773 - math/factor.cpp - 28 lines

```
mt19937_64 rng64(random_device{}());

using ull = unsigned long long;
using u128 = __uint128_t;

ull modMul(ull a, ull b, ull mod) { return (u128)a * b % mod;
↪ }

ull rho(ull n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    ull c = uniform_int_distribution<ull>(1, n - 1)(rng);
    ull x = uniform_int_distribution<ull>(0, n - 1)(rng), y =
    ↪ x, d = 1;
    auto f = [&](ull x) { return (modMul(x, x, n) + c) % n; };
    while (d == 1) {
        x = f(x);
        y = f(f(y));
        d = gcd(x > y ? x - y : y - x, n);
    }
    return d == n ? rho(n) : d;
} // a9272a

void factor(ull n, vector<ull> &fs) {
    if (n == 1) return;
    if (isPrime(n)) { fs.pb(n); return; }
    ull d = rho(n);
    factor(d, fs);
    factor(n / d, fs);
} // 6d58e9
```

Interpolação de Lagrange

`lagrange(xs, ys, x)` e `lagrange(ys, x)` calculam o valor do polinômio de lagrande no ponto `x`. A segunda função assume que os valores de `y` são dados para `x = 0, 1, ..., n - 1`. Complexidade de `lagrange(xs, ys, x)`: $O(n^2 \log(mod))$. Complexidade de `lagrange(ys, x)`: $O(n)$.

29e498 - math/lagrangian-interpolation.cpp - 26 lines

```
ll lagrange(const vector<ll> &xs, const vector<ll> &ys, ll x)
↪ {
    int n = sz(xs); ll ans = 0;
    rep(i,0,n) if (xs[i] == x) return ys[i];
    rep(i,0,n) {
        ll a = ys[i], b = 1;
        rep(j,0,n) if (i != j) a = a * (x - xs[j] + mod) %
        ↪ mod, b = b * (xs[i] - xs[j] + mod) % mod;
        ans = (ans + a * powm(b, mod - 2)) % mod;
    }
    return ans;
} // 51ae24

ll lagrange(const vector<ll> &y, ll x) {
    int n = sz(y);
    x = (x % mod + mod) % mod;
    if (x < n) return y[x];
    vector<ll> p(n + 1, 1), s(n + 1, 1), a(n);
    rep(i,0,n) p[i + 1] = p[i] * (x - i + mod) % mod;
    for (int i = n - 1; i >= 0; --i) s[i] = s[i + 1] * (x - i
    ↪ + mod) % mod;
    a[0] = 1;
    rep(i,1,n) a[i] = a[i - 1] * i % mod;
    a[n - 1] = powm(a[n - 1], mod - 2);
    for (int i = n - 1; i; --i) a[i - 1] = a[i] * i % mod;
    ll ans = 0;
    rep(i,0,n) ans = (ans + p[i] * s[i + 1] % mod * a[i] % mod
    ↪ * a[n - 1 - i] % mod * (((n - 1 - i) & 1) ? mod - y[i]
    ↪ : y[i])) % mod;
    return ans;
} // 05868c
```

Polinômio de Lagrange

`lagrangePoly(xs, ys)` retorna o polinômio que passa pelos pontos (x_i, y_i) para `i = 0, 1, ..., n - 1`. Complexidade: $O(n^2)$.

7d0710 - math/lagrangian-polynomial.cpp - 19 lines

```
vector<ll> lagrangePoly(const vector<ll> &xs, const vector<ll>
↪ &ys) {
    int n = sz(xs);
    vector<ll> p = {1}, f(n);
    rep(i,0,n) {
        p.pb(0);
        for (int j = i + 1; j; --j) p[j] = (p[j - 1] + mod -
        ↪ p[j] * xs[i] % mod) % mod;
        p[0] = p[0] * (mod - xs[i]) % mod;
    }
    rep(i,0,n) {
        vector<ll> q(n);
        q[n - 1] = p[n];
        for (int j = n - 1; j; --j) q[j - 1] = (p[j] + q[j] *
        ↪ xs[i]) % mod;
        ll den = 1;
        rep(j,0,n) if (i != j) den = den * (xs[i] - xs[j] +
        ↪ mod) % mod;
        ll c = ys[i] * powm(den, mod - 2) % mod;
        rep(j,0,n) f[j] = (f[j] + c * q[j]) % mod;
    }
    return f;
} // 7d0710
```

Berlekamp-Massey

Encontra os coeficientes da recorrência que forma `s`. Complexidade: $O(n^2)$.

68a616 - math/berlekamp-massey.cpp - 19 lines

```
vi berlekampMassey(vi s) {
    vi c(1, 1), b(1, 1);
    int l = 0, m = 1; ll bb = 1;
    rep (n, 0, sz(s)) {
        ll d = 0;
        rep (i, 0, l + 1) d = (d + 1LL * c[i] * s[n - i]) %
        ↪ mod;
        if (d < 0) d += mod;
        if (!d) { m++; continue; }
        vi t = c;
        ll x = d * powm(bb, mod - 2) % mod;
        if (sz(c) < sz(b) + m) c.resize(sz(b) + m);
        rep (i, 0, sz(b)) c[i + m] = (c[i + m] - x * b[i]) %
        ↪ mod;
        if (2 * l <= n) l = n + 1 - l, b = t, bb = d, m = 1;
        else m++;
    }
    c.erase(begin(c));
    for (int &x : c) x = (mod - x) % mod;
    return c;
} // 68a616
```

Recorrência linear

Encontra o k-ésimo termo da recorrência linear pelos primeiros termos e os coeficientes. Complexidade: $O(n^2 \log(k))$.

5f7ef4 - math/linear-recurrence.cpp - 23 lines

```
int linRec(vi s, vi tr, ll k) { // s = first m terms,
↪  0-indexed kth term
    int n = sz(tr);
    if (k < n) return s[k];
    auto mul = [&](vi a, vi b) {
        vi c(2 * n + 1);
        rep (i, 0, n + 1) rep (j, 0, n + 1)
            c[i + j] = (c[i + j] + 1LL * a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i) rep (j, 0, n)
            c[i - 1 - j] = (c[i - 1 - j] + 1LL * c[i] * tr[j])
            ↪  % mod;
        c.resize(n + 1);
        return c;
    };
    vi pol(n + 1), e(n + 1);
    pol[0] = 1;
    e[1] = 1;
    for (++k; k; k >>= 1) {
        if (k & 1) pol = mul(pol, e);
        e = mul(e, e);
    }
    ll ans = 0;
    rep (i, 0, n) ans = (ans + 1LL * pol[i + 1] * s[i]) % mod;
    return ans;
} // 5f7ef4
```

Estruturas de Dados

Segtree

Segtree iterativa genérica: `set(i, x)` e `query(l, r)` em $O(\log n)$.

b76e87 - dsa/segtree.cpp - 18 lines

```
template<class S, S (*op)(S, S), S (*e)()>
struct Segtree {
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e()), n(N) {}
    void set(int i, S value) {
        t[i += n] = value;
        for (i >>= 1; i; i >>= 1) t[i] = op(t[i << 1], t[i <<
        ↪  1 | 1]);
    } // 630e57
    S query(int l, int r) {
        S al = e(), ar = e();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 9ee903
};
```

Lazy Segtree

Lazy segtree genérica: `set`, `apply` e `sum` em $O(\log n)$.

21bbc4 - dsa/lazy-segtree.cpp - 50 lines

```
template<class S, S (*op)(S, S), S (*e)(), class U, S
↪  (*mapping)(U, S),
        U (*compose)(U, U), U (*id)()>
struct LazySegtree {
    vector<S> t;
    vector<U> d;
    int n;
    LazySegtree(int N): t(4 * N, e()), d(4 * N, id()), n(N) {}
    void applyNode(int i, U u) {
        t[i] = mapping(u, t[i]);
        d[i] = compose(u, d[i]);
    } // 3e5d0d
    void push(int i) {
        applyNode(i << 1, d[i]);
        applyNode(i << 1 | 1, d[i]);
        d[i] = id();
    } // 7836a8
    void pull(int i) { t[i] = op(t[i << 1], t[i << 1 | 1]); }
    void set(int j, S val) { set(j, val, 0, n, 1); }
    void set(int j, S val, int tl, int tr, int i) {
        if (tl + 1 == tr) {
            t[i] = val;
            d[i] = id();
            return;
        }
        push(i);
        int tm = (tl + tr) / 2;
        if (j < tm) set(j, val, tl, tm, i << 1);
        else set(j, val, tm, tr, i << 1 | 1);
        pull(i);
    } // f9c2fd
    void apply(int l, int r, U u) { apply(l, r, u, 0, n, 1); }
    void apply(int l, int r, U u, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) return applyNode(i, u);
        push(i);
        int tm = (tl + tr) / 2;
        apply(l, r, u, tl, tm, i << 1);
        apply(l, r, u, tm, tr, i << 1 | 1);
        pull(i);
    } // 3f0fb7
    S sum(int l, int r) { return sum(l, r, 0, n, 1); }
    S sum(int l, int r, int tl, int tr, int i) {
        if (r <= tl || tr <= l) return e();
        if (l <= tl && tr <= r) return t[i];
        push(i);
        int tm = (tl + tr) / 2;
        return op(sum(l, r, tl, tm, i << 1),
                  sum(l, r, tm, tr, i << 1 | 1));
    } // b976f6
};
```

Sparse Table

Estrutura estática para idempotentes: build $O(n \log n)$ e query $O(1)$.

2e4ed0 - dsa/sparse-table.cpp - 18 lines

```
template <class S, S (*op)(S, S)> struct SparseTable {
    vector<vector<S>> t;
    vi lg;
    void build(vector<S> &v) {
        int n = ssize(v);
        lg.assign(n + 1, 0);
        rep (i, 2, n + 1) lg[i] = lg[i >> 1] + 1;
        t.assign(lg[n] + 1, vector<S>(n));
        t[0] = v;
        rep (k, 1, lg[n] + 1)
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = op(t[k - 1][i], t[k - 1][i + (1 <<
                ↪  (k - 1))]);
    } // 9c11fe
    S query(int l, int r) {
        int k = lg[r - l];
        return op(t[k][l], t[k][r - (1 << k)]);
    } // d2d23e
};
```

Convolução

FFT e NTT para multiplicar polinômios em $O(n \log n)$.

FFT

c8b36c - dsa/fft.cpp - 38 lines

```
using cd = complex<ld>;
const ld PI = acosl(-1.0L);

void fft(vector<cd> &a) {
    int n = sz(a), lg = 31 - __builtin_clz(n);
    static vector<cd> rt(2, 1);
    for (static int k = 2; k < n; k <<= 1) {
        rt.resize(n);
        cd x = polar((ld)1, PI / k);
        rep(i, k, 2 * k) rt[i] = i & 1 ? rt[i / 2] * x : rt[i
        ↪  / 2];
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | ((i & 1) << lg)) / 2;
    rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            rep(j, 0, k) {
                cd z = rt[j + k] * a[i + j + k];
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
} // c30ab4

vector<ld> conv(vector<ld> &a, vector<ld> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    vector<cd> in(n), out(n);
    rep(i, 0, sz(a)) in[i].real(a[i]);
    rep(i, 0, sz(b)) in[i].imag(b[i]);
    fft(in);
    for (cd &x : in) x *= x;
    rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    vector<ld> res(s);
    rep(i, 0, s) res[i] = imag(out[i]) / (4 * n);
    return res;
} // 6d98ab
```

NTT

8e2497 - dsa/ntt.cpp - 40 lines

```
// const int mod = 998244353, ROOT = 62;
void ntt(vector<ll> &a) {
    int n = sz(a);
    static array<ll, 1 << 23> r = {1, 1};
    for (static int t = 2, s = 2; t < n; t <<= 1, s++) {
        array z = {1ll, powm(ROOT, mod >> s)};
        for (int i = t; i < t << 1; i++) r[i] = r[i >> 1] *
        ↪  z[i & 1] % mod;
    }
    static array<ll, 1 << 23> b, c;
    copy(all(a), b.begin());
    for (int m = n; m >>= 1;) {
        for (int l = 0; l < n; l += m << 1)
            rep (i, 0, m)
                c[l + i] = b[l + 2 * i], c[l + m + i] = b[l +
                ↪  2 * i + 1];
        copy(c.begin(), c.begin() + n, b.begin());
    }
    for (int m = 1; m < n; m <<= 1) {
        for (int l = 0; l < n; l += m << 1)
            rep (i, 0, m) {
                ll z = r[m + i] * b[l + m + i] % mod;
                c[l + i] = b[l + i] + z;
                c[l + m + i] = b[l + i] + (mod - z);
            }
        rep (i, 0, n) b[i] = c[i] < mod ? c[i] : c[i] - mod;
    }
    rep(i, 0, n) a[i] = b[i];
} // 6019c2

vector<ll> conv(vector<ll> &a, vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int s = sz(a) + sz(b) - 1, n = 1;
    while (n < s) n <<= 1;
    int inv = powm(n, mod - 2);
    vector<ll> L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    rep(i, 0, n) out[-i & (n - 1)] = L[i] * R[i] % mod * inv %
    ↪  mod;
    ntt(out);
    return {out.begin(), out.begin() + s};
} // ad22f1
```

Convolução de XOR

`xorConv(a, b)` para convolução por XOR em $O(n \log n)$.

fd3fd7 - dsa/xor-convolution.cpp - 20 lines

```
void fwht(vector<ll> &a) {
    for (int k = 1; k < sz(a); k <<= 1)
        for (int i = 0; i < sz(a); i += 2 * k)
            rep (j, i, i + k) {
                ll x = a[j], y = a[j + k];
                a[j] = x + y; // mod: a[j] = (x + y) % mod;
                a[j + k] = x - y; // mod: a[j + k] = (x - y +
                ↪  mod) % mod;
            }
} // ae23e1

vector<ll> xorConv(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < max(sz(a), sz(b))) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    rep (i, 0, n) a[i] *= b[i]; // mod: a[i] = a[i] * b[i] %
    ↪  mod;
    fwht(a);
    rep (i, 0, n) a[i] /= n; // mod: a[i] = a[i] * invN %
    ↪  mod;
    return a;
} // 0bc2c7
```

Zeta e Mobius

Transformadas em subconjuntos para vetores indexados por máscara, $O(B2^B)$.

eff0df - dsa/zeta.cpp - 28 lines

```
vi subsetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep (i, 1, n)
            if (i & j) a[i] += a[i ^ j];
    return a;
} // b39c8d
vi subsetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i] -= a[i ^ j];
    return a;
} // 5ffc86
vi supersetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] += a[i];
```

```
    return a;
} // c5d556
vi supersetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] -= a[i];
    return a;
} // 128136
```

Árvore de Li Chao

Envelope de retas para mínimo: `insert` e `query` em $O(\log N)$.

9e88be - dsa/li-chao.cpp - 29 lines

```
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} //
    ↪ Replace with LLONG_MIN in case of finding max
    void insert(Line line) { insert(line, 0, n, 1); }
    void insert(Line line, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (line(tm) < t[i](tm)) swap(line, t[i]); // Replace
        ↪ with > in case of finding max
        if (tl + 1 == tr) return;
        if (t[i](tl) < line(tl)) insert(line, tm, tr, i << 1 |
        ↪ 1); // Replace with > in case of finding max
        else insert(line, tl, tm, i << 1);
    } // f784a4
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN
        ↪ in case of finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with >
        ↪ in case of finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore de Li Chao++

Li Chao com retas ativas só em subintervalos, $O(\log^2 N)$.

e28ce0 - dsa/li-chao-extended.cpp - 36 lines

```
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} //
    ↪ Replace with LLONG_MIN in case of finding max
    void insert(Line line, int L, int R) { insert(line, L, R,
    ↪ 0, n, 1); }
    void insert(Line line, int L, int R, int tl, int tr, int
    ↪ i) {
        int tm = tl + (tr - tl) / 2;
        if (L >= tr || R <= tl) return;
        if (L <= tl && tr <= R) {
            if (line(tm) < t[i](tm)) swap(line, t[i]); //
            ↪ Replace with > in case of finding max
            if (tl + 1 == tr) return;
            if (t[i](tl) < line(tl)) insert(line, L, R, tm,
            ↪ tr, i << 1 | 1); // Replace with > in case of
            ↪ finding max
            else insert(line, L, R, tl, tm, i << 1);
        } else {
            if (tl + 1 == tr) return;
            insert(line, L, R, tm, tr, i << 1 | 1);
            insert(line, L, R, tl, tm, i << 1);
        }
    } // 3dcc38
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN
        ↪ in case of finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with >
        ↪ in case of finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore Cartesiana

`cartTree(n, le)` retorna o pai de cada índice em $O(n)$.

caeaba - dsa/cartesian-tree.cpp - 20 lines

```
vi cartTree(int n, auto le) {
    int last;
    vi st, par(n);
    rep (i, 0, n) {
        par[i] = i, last = -1;
        while (sz(st) && le(par[i], st.back())) {
            if (last != -1) par[last] = st.back();
            last = st.back();
            st.pop_back();
        }
        if (last != -1) par[last] = i;
        st.pb(i);
    }
    while (sz(st) > 1) {
        last = st.back();
        st.pop_back();
        par[last] = st.back();
    }
    return par;
} // caeaba
```

Treap implícita

Treap de sequência com `split`/`merge`; custo esperado $O(\log n)$.

ad716f - dsa/treap.cpp - 69 lines

```
struct Node;
using TN = Node*;

struct Node {
    int y = rng(), cnt, v;
    ll sum;
    bool rev = 0;
    TN l = 0, r = 0;
    Node(int v) : cnt(1), v(v), sum(v) {}
};
```

```
int cnt(TN t) { return t ? t->cnt : 0; }
ll sum(TN t) { return t ? t->sum : 0; }

void apply_rev(TN t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
} // 907c51
void push(TN t) {
    if (!t || !t->rev) return;
    apply_rev(t->l);
    apply_rev(t->r);
    t->rev = 0;
} // b46a2a
void pull(TN t) {
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->v + sum(t->l) + sum(t->r);
} // 6ee5eb

void split(TN t, int k, TN& a, TN& b) {
    if (!t) return a = b = 0, void();
    push(t);
    if (cnt(t->l) >= k) split(t->l, k, a, t->l), b = t;
    else split(t->r, k - cnt(t->l) - 1, t->r, b), a = t;
    pull(t);
} // b34b6d
TN merge(TN a, TN b) {
    if (!a || !b) return a ? a : b;
    if (a->y > b->y) return push(a), a->r = merge(a->r, b),
    ↪  pull(a), a;
    return push(b), b->l = merge(a, b->l), pull(b), b;
} // 529963

void ins(TN& t, int p, int v) {
    TN a, b;
    split(t, p, a, b);
    t = merge(merge(a, new Node(v)), b);
} // c2b578
void era(TN& t, int p) {
    TN a, b, c;
    split(t, p, a, b);
    split(b, 1, b, c);
    delete b;
    t = merge(a, c);
} // cf32a5
TN kth(TN t, int p) {
    while (t) {
        push(t);
        if (cnt(t->l) == p) return t;
        if (cnt(t->l) > p) t = t->l;
        else p -= cnt(t->l) + 1, t = t->r;
    }
    return 0;
} // 4ba488

void sep(TN t, int l, int r, TN& a, TN& b, TN& c) {
    split(t, r, b, c);
    split(b, l, a, b);
} // 4fa0ee
```

Simplex

Maximiza $c\dot{x}$ dado $Ax \leq b$.

becd4c - dsa/simplex.cpp - 57 lines

```
struct Simplex {
    static constexpr ld EPS = 1e-10;
    static constexpr ld INF = 1e100;
    int m, n;
    vi B, N;
    vector<vector<ld>> D;
    Simplex(vector<vector<ld>> A, vector<ld> b, vector<ld> c)
        : m(sz(b)), n(sz(c)), B(m), N(n + 1), D(m + 2,
        ↪  vector<ld>(n + 2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) B[i] = n + i, D[i][n] = -1, D[i][n + 1] =
        ↪  b[i];
        rep(j,0,n) N[j] = j, D[m][j] = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        rep(i,0,m+2) if (i != r)
            rep(j,0,n+2) if (j != s) D[i][j] -= D[r][j] *
            ↪  D[i][s] * inv;
        rep(j,0,n+2) if (j != s) D[r][j] *= inv;
        rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    } // eb9c6e
    bool simplex(int ph) {
        int x = ph == 1 ? m + 1 : m;
        while (1) {
            int s = -1;
            rep(j,0,n+1) if (!(ph == 2 && N[j] == -1) &&
                (s == -1 || D[x][j] < D[x][s] - EPS ||
                (abs(D[x][j] - D[x][s]) <= EPS && N[j] <
                ↪  N[s]))) s = j;
            if (D[x][s] >= -EPS) return 1;
            int r = -1;
            rep(i,0,m) if (D[i][s] > EPS &&
                (r == -1 || D[i][n + 1] / D[i][s] < D[r][n +
                ↪  1] / D[r][s] - EPS ||
                (abs(D[i][n + 1] / D[i][s] - D[r][n + 1] /
                ↪  D[r][s]) <= EPS && B[i] < B[r]))) r = i;
            if (r == -1) return 0;
            pivot(r, s);
        }
    } // fee585
    ld solve(vector<ld> &x) {
        int r = 0;
        rep(i,1,m) if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -EPS) {
            pivot(r, n);
            if (!simplex(1) || D[m + 1][n + 1] < -EPS) return
            ↪  -INF;
            rep(i,0,m) if (B[i] == -1) {
                int s = -1;
                rep(j,0,n+1) if (s == -1 || D[i][j] < D[i][s]
                ↪  - EPS ||
                    (abs(D[i][j] - D[i][s]) <= EPS && N[j] <
                    ↪  N[s])) s = j;
                pivot(i, s);
            }
        }
        if (!simplex(2)) return INF;
        x.assign(n, 0);
        rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n + 1];
        return D[m][n + 1];
    } // 38c01f
};
```

Grafos

Caminho Euleriano

`adj[v] = {to, id}`. Retorna a ordem de um caminho euleriano em $O(V + E)$.

483eba - graph/eulerian-walk.cpp - 23 lines

```
pair<vi, vi> eulerWalk(vector<vector<pii>>& g, int m, int s =
↪  0) {
    int n = sz(g);
    vi path, walk, d(n), vis(m), it(n);
    vector<pii> st = {{s, -1}};
    d[s]++; // remove to force cycles
    while (sz(st)) {
        auto [v, e] = st.back();
        if (it[v] == sz(g[v])) {
            path.pb(v), walk.pb(e);
```

```
            st.pop_back();
        } else {
            auto [u, id] = g[v][it[v]++];
            if (!vis[id]) {
                d[v]--, d[u]++;
                vis[id] = 1;
                st.eb(u, id);
            }
        }
    }
    for (int x : d) if (x < 0) return {{}, {}};
    if (sz(walk) != m + 1) return {{}, {}};
    return { vi(path.rbegin(), path.rend()), vi(walk.rbegin()
    ↪  + 1, walk.rend()) };
} // 483eba
```

Árvore de pontes

`comp[v]` = componente de `v` na árvore de pontes. Ponte `uv` sse `comp[u] != comp[v]`. $O(V + E)$.

c3d125 - graph/bridges.cpp - 31 lines

```
vi bridges(vector<vi> &g) {
    int n = sz(g), ti = 0, cc = 0;
    vi tin(n), low(n), st, comp;
    comp.assign(n, -1);
    auto make = [&](int v) {
        while (1) {
            int x = st.back(); st.pop_back();
            comp[x] = cc;
            if (x == v) break;
        }
        cc++;
    };
    auto dfs = [&](auto self, int v, int p) -> void {
        tin[v] = low[v] = ++ti;
        st.pb(v);
        int usedPar = 0;
        for (int to : g[v]) {
            if (to == p && !usedPar) { usedPar = 1; continue;
            ↪  }
            if (!tin[to]) {
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] > tin[v]) make(to);
            } else low[v] = min(low[v], tin[to]);
        }
    };
    rep(i,0,n) if (!tin[i]) {
        dfs(dfs, i, -1);
        make(i);
    }
    return comp;
} // c3d125
```

Árvore de bloco e corte

`t[i]` = a quais componentes biconexas o vértice `i` pertence. `t[i].size() > 1` quando `i` é um ponto de articulação. Quando `i >= n`, `t[i]` são os vértices na componente biconexa `i`. $O(V + E)$.

7605c4 - graph/block-cut-tree.cpp - 35 lines

```
vector<vi> bct(vector<vi> &g) {
    int n = sz(g), ti = 0;
    vector<vi> t(n);
    vi num(n), low(n), seen(n);
    vector<pii> st;
    auto add = [&](int a, int b) {
        int id = sz(t), it = id + 1;
        t.pb({});
        while (1) {
            auto [x, y] = st.back(); st.pop_back();
            if (seen[x] != it) seen[x] = it, t[x].pb(id),
            ↪  t[id].pb(x);
            if (seen[y] != it) seen[y] = it, t[y].pb(id),
            ↪  t[id].pb(y);
            if (x == a && y == b) break;
        }
    };
    auto dfs = [&](auto self, int v, int p) -> void {
        num[v] = low[v] = ++ti;
        int ch = 0;
        for (int to : g[v]) if (to != p) {
            if (!num[to]) {
                st.eb(v, to);
                ch++;
                self(self, to, v);
                low[v] = min(low[v], low[to]);
                if (low[to] >= num[v]) add(v, to);
            } else if (num[to] < num[v]) {
                st.eb(v, to);
                low[v] = min(low[v], num[to]);
            }
        }
        if (p == -1 && !ch) t.pb({v}), t[v].pb(sz(t) - 1);
    };
    rep(i, 0, n) if (!num[i]) dfs(dfs, i, -1);
    return t;
} // 7605c4
```

Dominator Tree

`domTree(g, s)` retorna o pai imediato; `-1` para inalcançáveis. Quase linear.

72f9c4 - graph/dominator-tree.cpp - 34 lines

```
vi domTree(vector<vi> &g, int s) {
    int n = sz(g), t = 0;
    vector<vi> rg(n + 1), bucket(n + 1);
    vi arr(n), rev(n + 1), par(n + 1), sdom(n + 1), dom(n +
    ↪  1), dsu(n + 1), mn(n + 1);
    auto dfs = [&](auto &&self, int u) -> void {
        arr[u] = ++t; rev[t] = u;
        sdom[t] = dom[t] = dsu[t] = mn[t] = t;
        for (int v : g[u]) {
            if (!arr[v]) self(self, v), par[arr[v]] = arr[u];
            rg[arr[v]].pb(arr[u]);
        }
    };
    auto find = [&](auto &&self, int u, int x = 0) -> int {
        if (u == dsu[u]) return x ? -1 : u;
        int v = self(self, dsu[u], x + 1);
        if (v < 0) return u;
        if (sdom[mn[dsu[u]]] < sdom[mn[u]]) mn[u] =
        ↪  mn[dsu[u]];
        return dsu[u] = v, x ? v : mn[u];
    };
    dfs(dfs, s);
    for (int i = t; i; i--) {
        for (int j : rg[i]) sdom[i] = min(sdom[i],
        ↪  sdom[find(find, j)]);
        if (i > 1) bucket[sdom[i]].pb(i);
        for (int j : bucket[i]) {
            int y = find(find, j);
            dom[j] = sdom[y] == sdom[j] ? sdom[j] : y;
        }
        if (i > 1) dsu[i] = par[i];
    }
    rep(i, 2, t + 1) if (dom[i] != sdom[i]) dom[i] =
    ↪  dom[dom[i]];
    vi idom(n, -1);
    rep(i, 1, t + 1) idom[rev[i]] = rev[dom[i]];
    return idom;
} // 72f9c4
```

Árvore geradora mínima direcionada rápida.

`dmst(n, r, es)` retorna o custo da arborescência mínima, ou -1; quase linear na prática.

540d7b - graph/fast-dmst.cpp - 72 lines

```
struct DE { int a, b; ll w; };
struct DN {
    DE e; ll d = 0;
    DN *l = 0, *r = 0;
    DN(DE e) : e(e) {}
};
void prop(DN *x) {
    if (!x || !x->d) return;
    x->e.w += x->d;
    if (x->l) x->l->d += x->d;
    if (x->r) x->r->d += x->d;
    x->d = 0;
} // e30db2
DN* meld(DN *a, DN *b) {
    if (!a || !b) return a ? a : b;
    prop(a), prop(b);
    if (a->e.w > b->e.w) swap(a, b);
    a->r = meld(a->r, b);
    swap(a->l, a->r);
    return a;
} // d8fb40
DN* pop(DN *a) { prop(a); return meld(a->l, a->r); }

ll dmst(int n, int r, vector<DE> es) {
    vector<DN*> h(n);
    for (auto e : es) if (e.a != e.b) h[e.b] = meld(h[e.b],
    ↪  new DN(e));
    vi uf(n), seen(n, -1), path;
    vector<ll> in(n);
    iota(all(uf), 0);
    auto find = [&](auto &&self, int x) -> int {
        return uf[x] == x ? x : uf[x] = self(self, uf[x]);
    };
    auto join = [&](int a, int b) {
        a = find(find, a), b = find(find, b);
        if (a == b) return 0;
        uf[b] = a;
        return 1;
    };
    seen[r] = r;
    ll ans = 0;
    rep (s, 0, n) {
        int u = find(find, s);
        if (u == r || seen[u] >= 0) continue;
        path.clear();
        while (seen[u] < 0) {
            path.pb(u);
            seen[u] = s;
            while (h[u] && find(find, h[u]->e.a) == find(find,
            ↪  h[u]->e.b))
                h[u] = pop(h[u]);
            if (!h[u]) return -1;
            prop(h[u]);
            ans += in[u] = h[u]->e.w;
            u = find(find, h[u]->e.a);
            if (seen[u] == s) {
                int v = u;
                DN *x = 0;
                while (1) {
                    int w = path.back();
                    path.pop_back();
                    h[w]->d -= in[w];
                    x = meld(x, h[w]);
                    join(v, w);
                    if (w == v) break;
                }
                u = find(find, v);
                h[u] = x;
                seen[u] = -1;
            }
        }
    }
    return ans;
} // 8b0ffd
```

Floresta de grafo funcional

`par[i]` = imagem de `i`. `root[i]` = representante do ciclo/componente de `i`. $O(n)$.

c1f247 - graph/functional-forest.cpp - 20 lines

```
struct FuncForest {
    vi par, root;
    FuncForest(vi f) : par(f), root(sz(f), -1) {
        vi deg(sz(f)), ord;
        for (int v : f) deg[v]++;
        queue<int> q;
        rep (i, 0, sz(f)) if (!deg[i]) q.push(i);
        while (sz(q)) {
            int v = q.front(); q.pop();
            ord.pb(v);
            if (!--deg[par[v]]) q.push(par[v]);
        }
        rep (i, 0, sz(f)) if (deg[i]) {
            root[i] = i;
            for (int v = par[i]; v != i; v = par[v]) root[v] =
            ↪  i, deg[v] = 0;
            deg[i] = 0;
        }
        for (int i = sz(ord) - 1; i >= 0; --i) root[ord[i]] =
        ↪  root[par[ord[i]]];
    }
};
```

Componentes fortemente conexas e grafo de condensação

`cnt` = número de SCCs. `comp[v]` = SCC de `v`. `condGraph` = DAG. $O(V + E)$.

2c057a - graph/scc.cpp - 31 lines

```
pair<int, vi> scc(vector<vi> &g) {
    int n = sz(g);
    vector<vi> rg(n);
    rep (v, 0, n) for (int u : g[v]) rg[u].pb(v);
    vi vis(n), ord, comp(n, -1);
    auto dfs1 = [&](auto self, int v) -> void {
        vis[v] = 1;
        for (int u : g[v]) if (!vis[u]) self(self, u);
        ord.pb(v);
    };
    auto dfs2 = [&](auto self, int v, int c) -> void {
        comp[v] = c;
        for (int u : rg[v]) if (comp[u] < 0) self(self, u, c);
    };
    rep (i, 0, n) if (!vis[i]) dfs1(dfs1, i);
    reverse(all(ord));
    int cnt = 0;
    for (int v : ord) if (comp[v] < 0) dfs2(dfs2, v, cnt++);
    return {cnt, comp};
} // a8c254

vector<vi> condGraph(vector<vi> &g, int cnt, vi &comp) {
    vector<vi> dag(cnt);
    rep (v, 0, sz(g)) for (int u : g[v])
        if (comp[v] != comp[u]) dag[comp[v]].pb(comp[u]);
    rep (i, 0, cnt) {
        sort(all(dag[i]));
        dag[i].erase(unique(all(dag[i])), end(dag[i]));
    }
    return dag;
} // a5cf62
```

2-SAT

`ts.clause(...)` adiciona cláusulas. `ts.solve()` decide. `ts.val[i]` = valor final da variável `i`. $O(V+E)$.

1412b8 - graph/2-sat.cpp - 22 lines

```
struct TwoSat {
    int n;
    vector<vi> g;
    vi val;
    TwoSat(int n) : n(n), g(2 * n) {}
    int lit(int x, bool v) { return 2 * x + v; }
    void imp(int a, int b) { g[a].pb(b); }
    void clause(int a, bool va, int b, bool vb) {
        int x = lit(a, va), y = lit(b, vb);
        imp(x ^ 1, y);
        imp(y ^ 1, x);
    } // 47d6a6
    bool solve() {
        auto [cnt, comp] = scc(g);
        val.assign(n, 0);
        rep (i, 0, n) {
            if (comp[2 * i] == comp[2 * i + 1]) return 0;
            val[i] = comp[2 * i] < comp[2 * i + 1];
        }
        return 1;
    } // a54ac0
};
```

Matching bipartido máximo

`hopKarp(g, l, r)` atualiza os matchings `l`/`r` e retorna o tamanho; $O(E\sqrt{V})$.

2d4485 - graph/hopcroft-karp.cpp - 31 lines

```
int hopKarp(vector<vi> &g, vi &l, vi &r) {
    int n = sz(l), m = sz(r);
    vi d(n);
    auto bfs = [&]() {
        queue<int> q;
        rep(i, 0, n) d[i] = l[i] < 0 ? q.push(i), 0 : -1;
        bool f = 0;
        while (sz(q)) {
            int v = q.front(); q.pop();
            for (int u : g[v]) {
                int w = r[u];
                if (w < 0) f = 1;
                else if (d[w] < 0) d[w] = d[v] + 1, q.push(w);
            }
        }
        return f;
    };
    auto dfs = [&](auto &&self, int v) -> bool {
        for (int u : g[v]) {
            int w = r[u];
            if (w < 0 || (d[w] == d[v] + 1 && self(self, w)))
            ↪ {
                l[v] = u, r[u] = v;
                return 1;
            }
        }
        return d[v] = -1, 0;
    };
    int ans = 0;
    while (bfs()) rep(i, 0, n) ans += l[i] < 0 && dfs(dfs, i);
    return ans;
} // 2d4485
```

Max Flow

Dinic

`addEdge(u, v, cap)` e `flow(s, t)` para fluxo máximo.

f40f68 - graph/dinic.cpp - 55 lines

```
struct Dinic {
    struct Edge {
        ll s, t, f, c, r;
    };
    vector<Edge> e;
    vector<vi> adj;
    vi ne, lvl;
    Dinic(int n): adj(n), ne(n), lvl(n) {}
    void addEdge(int a, int b, ll c) {
        adj[a].pb(sz(e));
        e.eb(a, b, 0, c, sz(e) + 1);
        adj[b].pb(sz(e));
        e.eb(b, a, 0, 0, sz(e) - 2);
    } // 598446
    bool bfs(int s, int t) {
        queue<int> q;
        q.push(s);
        fill(all(lvl), -1);
        lvl[s] = 0;
        while (sz(q)) {
            int x = q.front();
            q.pop();
            for (int i: adj[x]) {
                if (e[i].f == e[i].c) continue;
                if (lvl[e[i].t] != -1) continue;
                lvl[e[i].t] = lvl[x] + 1;
                q.push(e[i].t);
            }
        }
        return lvl[t] == -1;
    } // 1625d2
    ll dfs(int x, int t, ll f) {
        if (f == 0) return 0;
        if (x == t) return f;
        for (int &te = ne[x]; te < sz(adj[x]); te++) {
            int i = adj[x][te], y = e[i].t;
            if (lvl[y] != lvl[x] + 1) continue;
            if (ll df = dfs(y, t, min(f, e[i].c - e[i].f))) {
                e[i].f += df;
                e[i ^ 1].f -= df;
                return df;
            }
        }
        return 0;
    } // d7b05b
    ll flow(int s, int t, ll mx = LLONG_MAX) {
        ll mf = 0;
        while (true) {
            if (bfs(s, t)) break;
            fill(all(ne), 0);
            while (ll f = dfs(s, t, mx - mf)) mf += f;
        }
        return mf;
    } // 53644f
};
```

Min Cost Flow

MCMF

`addEdge(u, v, cap, cost)` e `flow(s, t)` para min-cost max-flow.

e07252 - graph/mcf.cpp - 58 lines

```
struct MCMF {
    struct E { int to, rev; ll cap, cost; };
    int n;
    vector<vector<E>> g;
    MCMF(int n) : n(n), g(n) {}
    void addEdge(int u, int v, ll cap, ll cost) {
        g[u].eb(v, sz(g[v]), cap, cost);
        g[v].eb(u, sz(g[u]) - 1, 0, -cost);
    } // 82791b
```

```
    pll flow(int s, int t, ll k = 4e18) {
        const ll INF = 4e18;
        ll fl = 0, cost = 0;
        vector<ll> p(n, INF), d(n), f(n);
        vi pv(n), pe(n), inq(n);
        queue<int> q;
        p[s] = 0, q.push(s), inq[s] = 1;
        while (sz(q)) {
            int u = q.front(); q.pop();
            inq[u] = 0;
            rep(i,0,sz(g[u])) {
                auto &e = g[u][i];
                if (e.cap && p[e.to] > p[u] + e.cost) {
                    p[e.to] = p[u] + e.cost;
                    if (!inq[e.to]) q.push(e.to), inq[e.to] =
                    ↪  1;
                }
            }
        }
        rep(i,0,n) if (p[i] == INF) p[i] = 0;
        while (fl < k) {
            fill(all(d), INF);
            priority_queue<pll, vector<pll>, greater<pll>> pq;
            d[s] = 0, f[s] = k - fl, pq.emplace(0, s);
            while (sz(pq)) {
                auto [du, u] = pq.top(); pq.pop();
                if (du != d[u]) continue;
                rep(i,0,sz(g[u])) {
                    auto &e = g[u][i];
                    if (!e.cap) continue;
                    ll nd = du + e.cost + p[u] - p[e.to];
                    if (d[e.to] > nd) {
                        d[e.to] = nd, pv[e.to] = u, pe[e.to] =
                        ↪  i;
                        f[e.to] = min(f[u], e.cap);
                        pq.emplace(nd, e.to);
                    }
                }
            }
            if (d[t] == INF) break;
            rep(i,0,n) if (d[i] < INF) p[i] += d[i];
            ll x = f[t];
            fl += x, cost += x * p[t];
            for (int v = t; v != s; v = pv[v]) {
                auto &e = g[pv[v]][pe[v]];
                e.cap -= x, g[v][e.rev].cap += x;
            }
        }
        return {fl, cost};
    } // e0d612
};
```

Matching bipartido com menor custo

`cost` = custo mínimo. `match[i]` = coluna escolhida para a linha `i`. $(n <= m)$ $O(n^2m)$.

c8c7c0 - graph/hungarian.cpp - 33 lines

```
pair<ll, vi> hungarian(vector<vector<ll>> a) {
    int n = sz(a), m = sz(a[0]);
    const ll INF = 4e18;
    vector<ll> u(n + 1), v(m + 1), minv(m + 1);
    vi p(m + 1), way(m + 1), ans(n);
    rep(i, 1, n + 1) {
        p[0] = i;
        int j0 = 0;
        fill(all(minv), INF);
        vector<char> used(m + 1);
        do {
            used[j0] = 1;
            int i0 = p[j0], j1 = 0;
            ll delta = INF;
            rep(j, 1, m + 1) if (!used[j]) {
                ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < minv[j]) minv[j] = cur, way[j] = j0;
                if (minv[j] < delta) delta = minv[j], j1 = j;
            }
            rep(j, 0, m + 1)
                if (used[j]) u[p[j]] += delta, v[j] -= delta;
                else minv[j] -= delta;
            j0 = j1;
        } while (p[j0]);
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;
        } while (j0);
    }
    rep(j, 1, m + 1) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans};
} // c8c7c0
```

Árvores

Base

Estrutura base com HLD: `addEdge` e `build(root)` em $O(n)$.

4c7dc0 - tree/base.cpp - 37 lines

```
struct Tree {
    int n, timer = 0;
    vector<vi> g;
    vi par, dep, cnt, head, in, out, rev;
    Tree(int n = 0)
        : n(n), g(n), par(n, -1), dep(n), cnt(n),
          head(n), in(n), out(n), rev(n) {}
    void addEdge(int u, int v) {
        g[u].pb(v);
        g[v].pb(u);
    } // 70fc01
    void dfsSz(int v, int p = -1) {
        par[v] = p;
        cnt[v] = 1;
        if (p != -1) g[v].erase(find(all(g[v]), p));
        for (int &u : g[v]) {
            dep[u] = dep[v] + 1;
            dfsSz(u, v);
            cnt[v] += cnt[u];
            if (cnt[u] > cnt[g[v][0]]) swap(u, g[v][0]);
        }
    } // 78db39
    void dfsHld(int v) {
        in[v] = timer;
        rev[timer++] = v;
        for (int u : g[v]) {
            head[u] = (u == g[v][0] ? head[v] : u);
            dfsHld(u);
        }
        out[v] = timer;
    } // 53c04a
    void build(int root = 0) {
        dfsSz(root);
        head[root] = root;
        dfsHld(root);
    } // 4540b8
};
```

Utilidades

Helpers sobre `Tree`: ancestral, LCA, distância e navegação em caminhos; em geral $O(\log n)$.

178120 - tree/helpers.cpp - 43 lines

```
bool isAnc(Tree& t, int a, int b) {
    return t.in[a] <= t.in[b] && t.out[b] <= t.out[a];
```

```
} // eb6def
pii subtree(Tree& t, int v) {
    return {t.in[v], t.out[v]};
} // 15c9eb
int lca(Tree& t, int a, int b) {
    while (t.head[a] != t.head[b]) {
        if (t.dep[t.head[a]] > t.dep[t.head[b]]) a =
        ↪  t.par[t.head[a]];
        else b = t.par[t.head[b]];
    }
    return t.dep[a] < t.dep[b] ? a : b;
} // ba5b42
int dist(Tree& t, int a, int b) {
    int c = lca(t, a, b);
    return t.dep[a] + t.dep[b] - 2 * t.dep[c];
} // e0c63e
int kthAnc(Tree& t, int v, int k) {
    while (v != -1) {
        int h = t.head[v], d = t.dep[v] - t.dep[h];
        if (k <= d) return t.rev[t.in[v] - k];
        k -= d + 1;
        v = t.par[h];
    }
    return -1;
} // f240ac
// k-th vertex on path a -> b, 0-indexed
int jump(Tree& t, int a, int b, int k) {
    int c = lca(t, a, b);
    int da = t.dep[a] - t.dep[c], db = t.dep[b] - t.dep[c];
    if (k < 0 || k > da + db) return -1;
    if (k <= da) return kthAnc(t, a, k);
    return kthAnc(t, b, da + db - k);
} // b9e7b6
void getPath(Tree& t, int a, int b, auto f) {
    while (t.head[a] != t.head[b]) {
        if (t.dep[t.head[a]] > t.dep[t.head[b]]) swap(a, b);
        f(t.in[t.head[b]], t.in[b] + 1);
        b = t.par[t.head[b]];
    }
    if (t.dep[a] > t.dep[b]) swap(a, b);
    f(t.in[a], t.in[b] + 1);
} // 6ee927
```

LCA

`LCA lca(g, root)` e `lca.lca(a, b);` build $O(n \log n)$, query $O(1)$.

1d44d9 - tree/lca.cpp - 23 lines

```
using RMQ = SparseTable<int, [](int a, int b) { return min(a,
↪  b); }>;

struct LCA {
    int t = 0;
    vi tm, path, ret;
    RMQ rmq;
    LCA(vector<vi> &g, int r = 0) : tm(g.size()) {
        dfs(g, r, -1);
        rmq.build(ret);
    }
    void dfs(vector<vi> &g, int x, int pre) {
        tm[x] = t++;
        for (int y: g[x]) if (y != pre) {
            path.pb(x), ret.pb(tm[x]);
            dfs(g, y, x);
        }
    } // 2e7ca5
    int lca(int x, int y) {
        if (x == y) return x;
        tie(x, y) = minmax(tm[x], tm[y]);
        return path[rmq.query(x, y)];
    } // e8105a
};
```

Matching em Árvores

`treeMatching(g, root)` retorna o parceiro de cada vértice em $O(n)$.

6e0886 - tree/tree-matching.cpp - 17 lines

```
vi treeMatching(vector<vi> &g, int root = 0) {
    int n = sz(g);
    vi par(n, -1), ord{root}, mat(n, -1);
    rep(i,0,sz(ord)) {
        int v = ord[i];
        for (int u : g[v]) if (u != par[v]) {
            par[u] = v;
            ord.pb(u);
        }
    }
    for (int i = n - 1; i > 0; --i) {
        int v = ord[i], p = par[v];
        if (mat[v] == -1 && mat[p] == -1)
            mat[v] = p, mat[p] = v;
    }
    return mat;
} // 6e0886
```

Strings

KMP

`kmp(s)` retorna a prefix function em $O(n)$.

cb52b3 - strings/kmp.cpp - 10 lines

```
vi kmp(string s) {
    int n = sz(s), len = 0;
    vi pre(n);
    rep (i, 1, n) {
        while (len > 0 && s[i] != s[len]) len = pre[len - 1];
        if (s[i] == s[len]) len++;
        pre[i] = len;
    }
    return pre;
} // cb52b3
```

Z-function

`zFunc(s)` retorna o tamanho do match com o prefixo em cada posição, em $O(n)$.

fc69f3 - strings/z-function.cpp - 12 lines

```
vi zFunc(string s) {
    int n = sz(s), l = 0, r = 0;
    vi z(n);
    if (!n) return z;
    rep(i, 1, n) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    z[0] = n;
    return z;
} // fc69f3
```

Array de sufixos

`sufArr(s)` retorna os índices dos sufixos em ordem lexicográfica em $O(n \log n)$.

f69f29 - strings/suffix-array.cpp - 27 lines

```
vi sufArr(string &s) { // Could also be vector<T> &s
    int n = sz(s);
    vi c(n), d(n), e(n), sb(n), sa(n), cnt(n + 1);
    iota(all(sa), 0);
    sort(all(sa), [&](int i, int j) { return s[i] < s[j]; });
    c[sa[0]] = 1;
    for (int i = 1; i < n; i++)
        c[sa[i]] = c[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    for (int k = 1; c[sa[n - 1]] != n; k <<= 1) {
        rep (i, 0, n) d[i] = i + k < n ? c[i + k] : 0;
        auto srt = [&](auto &C) {
            fill(all(cnt), 0);
            rep (i, 0, n) cnt[C[i] + 1]++;
            rep (i, 0, n) cnt[i + 1] += cnt[i];
            for (int i: sa) sb[cnt[C[i]]++] = i;
            swap(sa, sb);
        };
        srt(d); srt(c);
        e[sa[0]] = 1;
        rep(i,1,n) {
            int a = sa[i-1], b = sa[i];
            e[b] = e[a] + (c[a] != c[b] || d[a] != d[b]);
        }
        swap(c, e);
    }
    return sa;
} // f69f29
```

Maior prefixo comum (LCP)

`lcpArr(s, sa)` calcula o LCP entre sufixos adjacentes em $O(n)$.

558ffb - strings/lcp.cpp - 12 lines

```
vi lcpArr(string &s, vi &sa) {
    int n = sz(s), k = 0;
    vi rnk(n), lcp(n);
    rep(i,0,n) rnk[sa[i]] = i;
    rep(i,0,n) if (rnk[i]) {
        int j = sa[rnk[i] - 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k])
        ↪  k++;
        lcp[rnk[i]] = k;
        if (k) k--;
    }
    return lcp;
} // 558ffb
```

Árvore de sufixos

`ST(s, sa, lcp)` constrói a árvore de sufixos a partir de `sa` e `lcp` em $O(n)$.

3e9f4e - strings/suffix-tree.cpp - 31 lines

```
struct ST {
    struct N {
        int l, r, p, suf;
        vi ch;
    };
    vector<N> t;
    vi dep;
    int nn(int l, int r, int p, int suf = -1) {
        t.pb({l, r, p, suf, {}});
        dep.pb(p == -1 ? 0 : dep[p] + r - l);
        if (p != -1) t[p].ch.pb(sz(t) - 1);
        return sz(t) - 1;
    } // aa5bfe
    ST(string& s, vi& sa, vi& lcp) {
        int n = sz(s), rt = nn(0, 0, -1), last = rt;
        rep(i,0,n) {
            int x = sa[i], k = i ? lcp[i] : 0;
            while (dep[last] > k) last = t[last].p;
            if (dep[last] < k) {
                int y = t[last].ch.back(), z = nn(t[y].l,
                ↪  t[y].l + k - dep[last], last);
                t[last].ch.back() = z;
                t[y].p = z;
                t[y].l += k - dep[last];
                t[z].ch.pb(y);
                dep[z] = k;
                last = z;
            }
            last = nn(x + k, n, last, x);
        }
    }
};
```

Automato de sufixos

`SAM sam`; `extend(c)` monta o automato em $O(n)$ total.

a67d77 - strings/suffix-automaton.cpp - 40 lines

```
struct SAM {
    struct N {
        int len = 0, link = -1;
        array<int, 26> to;
        ll cnt = 0; // endpos count after calcCnt()
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    int last = 0;
    void extend(char c) {
        int x = c - 'a', cur = sz(t), p = last;
        t.eb();
        t[cur].len = t[last].len + 1;
        t[cur].cnt = 1;
        for (; p != -1 && t[p].to[x] == -1; p = t[p].link)
        ↪  t[p].to[x] = cur;
        if (p == -1) t[cur].link = 0;
        else {
            int q = t[p].to[x];
            if (t[p].len + 1 == t[q].len) t[cur].link = q;
            else {
                int cl = sz(t);
                t.pb(t[q]);
                t[cl].len = t[p].len + 1;
                t[cl].cnt = 0;
                for (; p != -1 && t[p].to[x] == q; p =
                ↪  t[p].link) t[p].to[x] = cl;
                t[q].link = t[cur].link = cl;
            }
        }
        last = cur;
    } // 1f27c6
    vi ord() {
        int mx = 0;
        for (auto &u : t) mx = max(mx, u.len);
        vi cnt(mx + 1), ord(sz(t));
        for (auto &u : t) cnt[u.len]++;
        rep(i,1,mx+1) cnt[i] += cnt[i - 1];
        for (int i = sz(t) - 1; i >= 0; i--)
        ↪  ord[--cnt[t[i].len]] = i;
        return ord;
    } // a8d426
};
```

Para obter o número de substrings distintas, basta calcular a soma de `t[i].len - t[t[i].link].len` por todo o automato. Para obter o tamanho dos conjuntos `endpos` de cada estado, basta propagar os `cnt` por suffix links. Lembre-se que o automato de sufixos é um `DAG`.

Encontrando os palíndromos máximos em uma string

`manacher(s)` retorna `{d1, d2}` com raios ímpar/par em $O(n)$.

59be6a - strings/manacher.cpp - 17 lines

```
pair<vi, vi> manacher(string& s) {
    int n = sz(s);
    vi d1(n), d2(n);
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i +
        ↪  k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = i > r ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] ==
        ↪  s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return {d1, d2}; // {odd, even}
} // 59be6a
```

Aho-Corasick

`AC`: `add`, `build` e `next` para alfabeto 'a'-'z'. Construção/consulta lineares no total.

05f5f7 - strings/aho-corasick.cpp - 38 lines

```
struct AC {
    static constexpr int A = 26;
    struct N {
        array<int, A> to{};
        int link = 0;
        N() { to.fill(-1); }
    };
    vector<N> t{{}};
    vi ord;
    int add(string& s) {
        int v = 0;
        for (char c : s) {
            int x = c - 'a';
            if (t[v].to[x] == -1) t[v].to[x] = sz(t), t.eb();
            v = t[v].to[x];
        }
        return v;
    } // bd83ba
    void build() {
        queue<int> q;
        rep(c,0,A) {
            int u = t[0].to[c];
            if (u == -1) t[0].to[c] = 0;
            else q.push(u);
        }
        ord.clear();
        while (sz(q)) {
            int v = q.front(); q.pop();
            ord.pb(v);
            rep(c,0,A) {
                int& u = t[v].to[c];
                if (u == -1) u = t[t[v].link].to[c];
                else t[u].link = t[t[v].link].to[c],
                ↪  q.push(u);
            }
        }
    } // 5e19d4
    int next(int v, char c) { return t[v].to[c - 'a']; }
};
```

Geometria

Base

Base com `Pt`, `xx/yy`, `dot`, `cross`, `perp`, `EPS` e `sgn`.

673f55 - geometry/core.cpp - 9 lines

```
using Pt = complex<ll>;
#define xx real()
#define yy imag()

ll dot(Pt a, Pt b) { return (conj(a) * b).xx; }
ll cross(Pt a, Pt b) { return (conj(a) * b).yy; }
Pt perp(Pt a) { return Pt(-a.yy, a.xx); }
const ld EPS = 1e-9;
int sgn(ld x) { return (x > EPS) - (x < -EPS); }
```

Interseção de retas

`lineInter(a, b, c, d)` para retas dadas por dois pontos; $O(1)$.

2fc473 - geometry/line-intersection.cpp - 3 lines

```
Pt lineInter(Pt a, Pt b, Pt c, Pt d) {
    return a + (b - a) * cross(c - a, d - c) / cross(b - a, d
    ↪  - c);
} // 2fc473
```

Interseção de segmentos

`segInter` e `properInter` para dois segmentos, em $O(1)$.

f6de72 - geometry/segment-intersection.cpp - 16 lines

```
bool segInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b -
    ↪  a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d -
    ↪  c, b - c));
    if (ab_c * ab_d < 0 && cd_a * cd_b < 0) return 1;
    if (!ab_c && sgn(dot(c - a, c - b)) <= 0) return 1;
    if (!ab_d && sgn(dot(d - a, d - b)) <= 0) return 1;
    if (!cd_a && sgn(dot(a - c, a - d)) <= 0) return 1;
    if (!cd_b && sgn(dot(b - c, b - d)) <= 0) return 1;
    return 0;
} // 4f106a

bool properInter(Pt a, Pt b, Pt c, Pt d) {
    int ab_c = sgn(cross(b - a, c - a)), ab_d = sgn(cross(b -
    ↪  a, d - a));
    int cd_a = sgn(cross(d - c, a - c)), cd_b = sgn(cross(d -
    ↪  c, b - c));
    return ab_c * ab_d < 0 && cd_a * cd_b < 0;
} // 66a13a
```

Área de um polígono

`polyArea(p)` retorna a área orientada multiplicada por 2, em $O(n)$.

b23e06 - geometry/polygon-area.cpp - 5 lines

```
ll polyArea(vector<Pt> &pt) { // ld: change ret type to ld
    ll n = sz(pt), s = 0; // ld: change to ld s
    rep (i, 0, n) s += cross(pt[i], pt[(i + 1) % n]);
    return s;
} // b23e06
```

Verificar se um ponto está dentro de um polígono

Point-in-convex-polygon em $O(\log n)$.

1ce015 - geometry/point-in-polygon.cpp - 10 lines

```
bool pointInPoly(vector<Pt> &pt, Pt q, bool border = true) {
    int n = pt.size(), t = border, l = 1, r = n - 1; // ld: ld
    ↪  t = border ? EPS : 0
    auto v = [&](int i) { return cross(q - pt[0], pt[i] -
    ↪  pt[0]); };
    if (v(l) >= t || v(r) <= -t) return 0;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (v(m) > 0 ? r : l) = m; // ld: use v(m) > EPS
    }
    return cross(pt[r] - pt[l], pt[l] - q) < t; // ld: use the
    ↪  ld t above
} // 1ce015
```

Convex hull

`convHull(pt)` retorna os índices dos vértices do hull em $O(n \log n)$.

b43fd9 - geometry/convex-hull.cpp - 22 lines

```
vi convHull(vector<Pt> &pt) {
    int n = sz(pt), m;
    vi h, ord(n);
    auto add = [&]() {
        vi st;
        for (int i: ord) {
            while ((m = sz(st)) > 1) {
                Pt a = pt[st[m - 1]], b = pt[st[m - 2]], c =
                ↪  pt[i];
                if (cross(b - a, c - a) < 0) break; // 1) >
                ↪  for clockwise, <= to include non-vertices
                st.pop_back();
            }
            st.pb(i);
        }
        st.pop_back();
        h.insert(h.end(), all(st));
    };
    iota(all(ord), 0);
    auto top = [](auto a) { return pair(a.xx, a.yy); };
    sort(all(ord), [&](int i, int j) { return top(pt[i]) >
    ↪  top(pt[j]); });
    add(), reverse(all(ord)), add();
    return h;
} // b43fd9 // 1) ld: use sgn(cross(...)) < 0
```

Menor distância euclideana

Closest pair: retorna um par de índices em $O(n \log n)$.

264831 - geometry/closest-pair.cpp - 33 lines

```
ll dist2(Pt a, Pt b) { // ld: ll -> ld
    ll dx = a.xx - b.xx, dy = a.yy - b.yy; // ld: ll -> ld
    return dx * dx + dy * dy;
} // ed4927

pii closestPair(vector<Pt> p) {
    vector<pair<Pt,int>> a;
    rep(i,0,sz(p)) a.eb(p[i], i);
    sort(all(a), [](auto a, auto b) {
        return a.fi.xx != b.fi.xx ? a.fi.xx < b.fi.xx :
        ↪  a.fi.yy < b.fi.yy;
    });
    ll ans = dist2(a[0].fi, a[1].fi), l = 0;
    pii best = {a[0].se, a[1].se};
    set<tuple<ll,ll,int>> s; // ld: use tuple<ld, ld, int>
    s.insert({a[0].fi.yy, a[0].fi.xx, a[0].se});
    s.insert({a[1].fi.yy, a[1].fi.xx, a[1].se});
    rep (i, 2, sz(a)) {
        ll d = (ll)sqrtl((ld)ans) + 1; // ld: ld d =
        ↪  sqrtl(ans) + EPS
        while (a[i].fi.xx - a[l].fi.xx > d) {
            s.erase({a[l].fi.yy, a[l].fi.xx, a[l].se});
            l++;
        }
        auto it1 = s.lower_bound({a[i].fi.yy - d, LLONG_MIN,
        ↪  INT_MIN});
        auto it2 = s.upper_bound({a[i].fi.yy + d, LLONG_MAX,
        ↪  INT_MAX});
        for (auto it = it1; it != it2; ++it) {
            int j = get<2>(*it);
            ll cur = dist2(a[i].fi, p[j]); // ld: ld cur =
            ↪  ...
            if (cur < ans) ans = cur, best = {a[i].se, j};
        }
        s.insert({a[i].fi.yy, a[i].fi.xx, a[i].se});
    }
    return best;
} // 130018
```

Interseção de círculo e reta

`circLineInter(c, r, a, d)` retorna 0, 1 ou 2 pontos, em $O(1)$.

21e90b - geometry/circle-line.cpp - 8 lines

```
vector<Pt> circLineInter(Pt c, ld r, Pt a, Pt d) {
    Pt f = a + d * dot(c - a, d) / norm(d);
    ld h2 = r * r - norm(c - f);
    if (sgn(h2) < 0) return {};
    ld dt = sqrt(max((ld)0, h2)) / abs(d);
    if (!sgn(h2)) return {f};
    return {f - dt * d, f + dt * d};
} // 21e90b
```

Interseção de círculos

`circCircInter(c1, r1, c2, r2)` retorna 0, 1 ou 2 pontos, em $O(1)$.

d3b1f8 - geometry/circle-circle.cpp - 12 lines

```
vector<Pt> circCircInter(Pt c1, ld r1, Pt c2, ld r2) {
    Pt d = c2 - c1;
    ld dist = abs(d);
    if (!sgn(dist)) return {};
    ld a = (norm(d) + r1 * r1 - r2 * r2) / (2 * dist);
    ld h2 = r1 * r1 - a * a;
    if (sgn(h2) < 0) return {};
    Pt mid = c1 + d * (a / dist);
    if (!sgn(h2)) return {mid};
    Pt p = perp(d) * (sqrt(max((ld)0, h2)) / dist);
    return {mid - p, mid + p};
} // d3b1f8
```

Tangentes comuns a dois círculos

`circTangents(c1, r1, c2, r2)` retorna pares de tangência, em $O(1)$.

0d00d5 - geometry/circle-tangents.cpp - 14 lines

```
vector<pair<Pt, Pt>> circTangents(Pt c1, ld r1, Pt c2, ld r2)
↪  {
    vector<pair<Pt, Pt>> res;
    for (int s: {1, -1}) {
```

```
        ld r = s * r2, dr = r1 - r;
        Pt d = c2 - c1;
        ld l2 = norm(d), h2 = max((ld)0, l2 - dr * dr);
        if (sgn(l2 - dr * dr) < 0) continue;
        for (int t: {1, -1}) {
            Pt n = (dr * d + perp(d) * (t * sqrt(h2))) / l2;
            res.pb({c1 + r1 * n, c2 + r * n});
        }
    }
    return res;
} // 0d00d5
```

Menor cobertura circular

Encontra o menor círculo que contém todos os pontos.

fd30f4 - geometry/enclosing-circle.cpp - 33 lines

```
struct C {
    Pt o;
    ld r;
};

bool in(C c, Pt p) { return abs(p - c.o) <= c.r + EPS; }

C circ(Pt a, Pt b) { return {(a + b) / (ld)2, abs(a - b) / 2};
↪  }

C circ(Pt a, Pt b, Pt c) {
    if (fabsl(cross(b - a, c - a)) < EPS) {
        C x = circ(a, b), y = circ(a, c), z = circ(b, c);
        if (in(x, c)) return x;
        if (in(y, b)) return y;
        return z;
    }
    Pt o = a + perp((b - a) * norm(c - a) - (c - a) * norm(b -
    ↪  a))
        / cross(b - a, c - a) / (ld)2;
    return {o, abs(a - o)};
} // a6aab7

C mec(vector<Pt> p) {
    shuffle(all(p), rng);
    C c{{0, 0}, -1};
    rep(i,0,sz(p)) if (!in(c, p[i])) {
        c = {p[i], 0};
        rep(j,0,i) if (!in(c, p[j])) {
            c = circ(p[i], p[j]);
            rep(k,0,j) if (!in(c, p[k])) c = circ(p[i], p[j],
            ↪  p[k]);
        }
    }
    return c;
} // effcbe
```

Soma Minkowski de polígonos convexos

`minkowski(a, b)` para convexos em sentido anti-horário, em $O(|a| + |b|)$.

107f6b - geometry/minkowski.cpp - 17 lines

```
bool ptLt(Pt a, Pt b) { return a.xx != b.xx ? a.xx < b.xx :
↪  a.yy < b.yy; } // 1)
vector<Pt> minkowski(vector<Pt> a, vector<Pt> b) {
    auto norm = [&](vector<Pt>& p) {
        rotate(p.begin(), min_element(all(p), ptLt), p.end());
        p.pb(p[0]), p.pb(p[1]);
    };
    int n = sz(a), m = sz(b);
    norm(a), norm(b);
    vector<Pt> c;
    for (int i = 0, j = 0; i < n || j < m; ) {
        c.pb(a[i] + b[j]);
        ll z = cross(a[i + 1] - a[i], b[j + 1] - b[j]); // ld:
        ↪  keep z in ld
        if (z >= 0 && i < n) i++; // ld: use z >= -EPS
        if (z <= 0 && j < m) j++; // ld: use z <= EPS
    }
    return c;
} // f04989 // 1) ld: a.xx != b.xx -> sgn(a.xx - b.xx)
```

Corte de Polígono

Corte de polígono por semiplano em $O(n)$.

a60a48 - geometry/polygon-cut.cpp - 10 lines

```
vector<Pt> polyCut(vector<Pt> p, Pt a, Pt b) {
    vector<Pt> q;
    rep(i,0,sz(p)) {
        Pt c = p[i], d = p[(i + 1) % sz(p)];
        int sc = sgn(cross(b - a, c - a)), sd = sgn(cross(b -
        ↪  a, d - a));
        if (sc >= 0) q.pb(c);
        if (sc * sd < 0) q.pb(lineInter(a, b, c, d));
    }
    return q;
} // a60a48
```

Fórmulas, Teoremas e Fatos

Combinatória e Sequências

Progressão aritmética.

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

$$a + (a+d) + \cdots + (a + (n-1)d) = \frac{n(2a + (n-1)d)}{2}$$

Progressão geométrica.

$$1 + r + r^2 + \cdots + r^n = \frac{r^{n+1} - 1}{r - 1} \qquad (r \neq 1)$$

$$a + ar + \cdots + ar^{n-1} = a\frac{r^n - 1}{r - 1} \qquad (r \neq 1)$$

Somas de potências comuns.

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

Coeficiente de multiconjunto.

$$\left(\!\binom{n}{k}\!\right) = \binom{n+k-1}{k}$$

Número de formas de escolher k elementos entre n tipos, com repetição.

Estrelas e Barras. O número de soluções inteiras não negativas de

$$x_1 + \cdots + x_k = n$$

é

$$\binom{n+k-1}{k-1}.$$

O número de soluções inteiras positivas é

$$\binom{n-1}{k-1}.$$

Estrelas e barras com cotas inferiores. O número de soluções inteiras de

$$x_1 + \cdots + x_n = S, \qquad x_i \geq l_i$$

é

$$\binom{S - \sum l_i + n - 1}{n-1},$$

após a substituição $y_i = x_i - l_i$.

Número de Catalan.

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

Desarranjos.

$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}$$

e

$$D_n = (n-1)(D_{n-1} + D_{n-2})$$

Convolução binomial (Vandermonde).

$$\sum_k \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n}.$$

Teorema do Casamento de Hall. Um grafo bipartido (L, R) tem um matching saturando L sse

$$\forall S \subseteq L, \quad |N(S)| \geq |S|.$$

Teorema de König. Em um grafo bipartido,

$$\text{matching máximo} = \text{cobertura mínima}.$$

Teorema de Dilworth. Em qualquer poset finito, o tamanho máximo de uma anticadeia é igual ao número mínimo de cadeias necessárias para cobrir todos os elementos.

Teorema de Mirsky. Em qualquer poset finito, o tamanho máximo de uma cadeia é igual ao número mínimo de anticadeias necessárias para cobrir todos os elementos.

Erdős–Szekeres. Toda sequência de $(r-1)(s-1)+1$ números distintos contém ou uma subsequência crescente de tamanho r, ou uma subsequência decrescente de tamanho s.

Teoria dos Números

Truques de floor/ceiling.

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a+b-1}{b} \right\rfloor \qquad (a, b > 0)$$

$$\left\lceil \frac{a}{b} \right\rceil = \frac{a+b-1}{b}$$

em aritmética inteira, para $a, b > 0$.

Contagem / soma de divisores. Se

$$n = \prod p_i^{e_i},$$

então

$$d(n) = \prod (e_i + 1),$$

$$\sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

Triplas pitagóricas. Todas as soluções inteiras primitivas de

$$a^2 + b^2 = c^2$$

são exatamente

$$a = m^2 - n^2, \qquad b = 2mn, \qquad c = m^2 + n^2,$$

para inteiros $m > n \geq 1$ com

$$\gcd(m, n) = 1 \quad \text{e} \quad m \not\equiv n \pmod 2.$$

Todas as soluções inteiras são obtidas multiplicando uma solução primitiva por $k \geq 1$.

Teorema das três somas de quadrados. Um inteiro não negativo n pode ser escrito como

$$n = x^2 + y^2 + z^2$$

com x, y, z inteiros sse

$$n \neq 4^a(8b + 7)$$

para todos os inteiros $a, b \geq 0$.

Fórmula de Legendre.

$$v_p(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{p^k} \right\rfloor$$

Teorema de Lucas. Para primo p, se

$$n = \sum n_i p^i, \qquad k = \sum k_i p^i,$$

então

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod p.$$

Teorema de Kummer. O expoente de um primo p em $\binom{n}{k}$ é igual ao número de vai-uns ao somar k e $n-k$ na base p.

LTE (versão comum). Se p é um primo ímpar, $p \mid (a-b)$ e $p \nmid ab$, então

$$v_p(a^n - b^n) = v_p(a - b) + v_p(n).$$

Além disso, para n ímpar e $p \mid (a+b)$,

$$v_p(a^n + b^n) = v_p(a + b) + v_p(n).$$

Inversão de Möbius. Se

$$g(n) = \sum_{d \mid n} f(d),$$

então

$$f(n) = \sum_{d \mid n} \mu(d)\, g\left(\frac{n}{d}\right).$$

Grafos e Jogos

Condições para trilha/circuito euleriano (direcionado). Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente fracamente conexa. Então:

- existe circuito euleriano sse $\deg^+(v) = \deg^-(v)$ para todo vértice;
- existe trilha euleriana sse exatamente um vértice tem $\deg^+(v) = \deg^-(v)+1$, exatamente um tem $\deg^-(v) = \deg^+(v) + 1$, e todos os outros são balanceados.

Condições para trilha/circuito euleriano (não direcionado). Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente conexa. Então:

- existe circuito euleriano sse todo vértice tem grau par;
- existe trilha euleriana sse exatamente 0 ou 2 vértices têm grau ímpar.

Critério de 2-SAT. Uma instância de 2-SAT é satisfatível sse, para toda variável x, os literais x e $\neg x$ estão em SCCs diferentes do grafo de implicação.

Restrições de diferença $\leftrightarrow$ menores caminhos. Um sistema de desigualdades da forma

$$x_v - x_u \leq w$$

pode ser modelado por uma aresta $u \to v$ de peso w. Então:

- o sistema é viável sse não existe ciclo negativo;
- uma solução viável é dada pelas distâncias a partir de uma super-fonte.

Cobertura por caminhos em DAG via matching. O número mínimo de caminhos disjuntos em vértices que cobrem todos os vértices de um DAG é

$$n - (\text{matching máximo na expansão bipartida do DAG}).$$

Teorema Matrix-Tree de Kirchhoff. O número de árvores geradoras de um grafo é igual a qualquer cofator de sua matriz Laplaciana.

Intervalos, Prefixos e Contribuições

Máximo número de intervalos disjuntos. Ordenar os intervalos por extremo direito crescente e escolher gulosamente produz um conjunto máximo de intervalos dois a dois disjuntos.

Mínimo de pontos para perfurar intervalos. Ordenar os intervalos por extremo direito crescente e escolher repetidamente o extremo direito do primeiro intervalo ainda não coberto produz um conjunto mínimo de pontos que intersecta todos os intervalos.

Intervalos: mínimo de pontos = máximo de disjuntos. Para uma família de intervalos na reta, o número mínimo de pontos necessários para intersectar todos os intervalos é igual ao número máximo de intervalos dois a dois disjuntos.

Intervalos dois a dois intersectantes têm interseção comum. Para intervalos na reta, se todo par intersecta, então todos compartilham um ponto em comum. Isso falha em dimensões maiores, mas é muito útil em 1D.

Strings

Teorema de Fine–Wilf. Se uma string tem períodos p e q, e seu tamanho é pelo menos

$$p + q - \gcd(p, q),$$

então ela também tem período $\gcd(p, q)$.

Substrings distintas. Usando suffix array:

$$\#\text{substrings distintas} = \frac{n(n+1)}{2} - \sum lcp[i]$$

Usando suffix automaton:

$$\#\text{substrings distintas} = \sum_{v \neq root} (len(v) - len(link(v)))$$

Geometria

Transformações da distância Manhattan. Em 2D,

$$L_1 = \max(|\Delta_+|, |\Delta_-|)$$

$$\Delta_\pm = (x \pm y)' - (x \pm y)$$

Na prática:

$$\max L_1 = \max(D_+, D_-)$$

$$D_\pm = \max(x \pm y) - \min(x \pm y)$$

Teorema de Pick. Para um polígono simples na grid,

$$A = I + \frac{B}{2} - 1,$$

onde I é o número de pontos de grid internos e B é o número de pontos de grid na fronteira.

Pontos de grid em um segmento. O número de pontos de grid no segmento de (x_1, y_1) até (x_2, y_2) é

$$\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1.$$

Fórmula de Heron. Para lados a, b, c e semiperímetro

$$s = \frac{a+b+c}{2},$$

a área do triângulo é

$$\sqrt{s(s-a)(s-b)(s-c)}.$$

Raio da circunferência circunscrita e inscrita. Para área do triângulo Δ:

$$R = \frac{abc}{4\Delta}, \qquad r = \frac{\Delta}{s}$$

onde $s = \frac{a+b+c}{2}$.

DP

Formular DP Se um problema pode ser resolvido com DP, mesmo que a solução óbvia resultaria em TLE, escreva a formulação da DP. Talvez seja possível otimizar a DP e resolver rápido o suficiente, ou mesmo generalizar a forma obtida para algo mais simples/fácil de calcular.

Otimizar com matrizes Especialmente útil quando o problema pode ser resolvido com uma DP que depende apenas de uma janela de estados anteriores de tamanho fixo. Ten-

tar representar as transições da DP como multiplicação de matrizes.

Otimizar com funções Pode ser que a DP tome o formato de uma função. Neste caso, talvez seja possível calcular o valor da DP em outros pontos descobrindo a forma da função, como usando interpolação ou outra forma.

Debugging

Leitura e modelagem.

- releia o enunciado, as amostras e as restrições de canto;
- sua interpretação das operações e da ordem está correta?;
- a resposta pede valor exato, no máximo, no mínimo, distintos ou módulo?

Limites e tipos.

- testou $n = 0$, $n = 1$, todos iguais, estritamente crescente/decrescente?;
- `ll` basta? precisa de `__int128` para multiplicações?;
- o `INF` é grande o bastante? `INF + x` pode estourar?;
- o módulo está sendo normalizado após subtrações?

Índices e intervalos.

- o código está em 0-index ou 1-index?;
- distinguiu $[l, r]$, $[l, r)$ e comprimento de caminho vs. número de vértices?;
- existe off-by-one em laços, prefix sums ou busca binária?;
- alocou tamanho n ou $n + 1$?

Inicialização.

- limpou todos os globais e casos de teste?;
- `dp`, `vis`, `dist` e `par` foram inicializados corretamente?;
- resetou grafo, DSU, segtree e fenwick em multitestcase?

Grafos e árvores.

- o grafo é direcionado ou não direcionado?;
- adicionou as duas arestas quando precisava?;
- self-loops e multi-edges importam?;
- tratou o caso desconexo?;
- a profundidade de recursão cabe ou precisa de versão iterativa?;
- as convenções de raiz, pai e profundidade estão consistentes?

Busca binária.

- o predicado é monótono?;
- o significado de `lo` e `hi` como viável/inviável está claro?;
- o meio e a condição de parada estão corretos?;
- você está retornando o primeiro `true` ou o último `true`?

Estruturas de dados.

- a compressão de coordenadas foi feita de forma consistente?;
- a lazy propagation faz push/pull corretamente?;
- em `multiset`, era para apagar um só ou todos?;
- existe invalidação de iteradores?

Matemática.

- a divisão é floor ou truncada para zero?;
- o módulo negativo foi tratado?;
- `gcd(0,0)` apareceu? o inverso realmente existe?;
- existe problema de precisão? precisa usar `ld` ou `eps`?

Implementação.

- o comparador de ordenação é estrito?;
- faltou `sort`, `unique` ou `reverse`?;
- está usando referências inválidas ou objetos movidos?;
- apareceu um `O(n^2)` ou `O(nm)` acidental?

Final.

- teste casos minúsculos na mão;
- teste casos extremos;
- confira invariantes importantes com `assert`;
- compare com brute em casos aleatórios pequenos.